Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

GENERATIVE AI

2026–27, Term-2

ABSTRACT

Section / Team Details

Section No.: 7

Team No.: 15

Project Title: Prompting vs RAG vs Fine-Tuning for Customer Support Automation

Team Members

2520030069-G.Hemaswi
2520030091-K.Rupesh

1. Abstract

Generative AI has moved quickly from research labs into everyday business tools, and customer support is one of its most common uses. Large Language Models (LLMs) are foundation models trained on huge text collections through next-token prediction, and they can draft replies, summarise tickets and answer product questions in natural language. This project studies **how a support team should adapt such a model to its own business: by Prompting, by Retrieval-Augmented Generation (RAG), or by Fine-Tuning.**

Module I gives the groundwork. It places Generative AI within the wider landscape of AI, Machine Learning, Deep Learning and NLP, and separates generative tasks (writing a reply) from discriminative tasks (classifying a ticket's intent or urgency). It also covers modalities (text, speech and images), foundation models, and the choice between open-weight and closed models, which affects control, data privacy and deployment cost in a support setting.

The module also covers the limits of LLMs that matter most for support. **Hallucinations** can produce confident but wrong answers about refunds or policies, **knowledge cutoffs** leave a model unaware of new products and prices, and **pricing, rate limits and failure rates** decide whether a solution can handle real ticket volumes. Prompting is the cheapest to set up but relies on what the model already knows. RAG grounds answers in retrieved company documents and helps with cutoffs and hallucinations. Fine-tuning changes the model itself to fit a tone, format or domain, but needs training data and effort.

A second focus is **tokenization**. LLMs read tokens, not words: text is split into tokens by methods such as BPE, WordPiece and SentencePiece, mapped to token IDs through a fixed vocabulary, and processed from there. This matters for a multilingual market. English text usually needs fewer tokens than Telugu or Hindi, whose scripts are often split into many more pieces. The same support message can therefore cost more and use more of the context window when written in an Indian language, which makes the comparison of the three approaches in English, Telugu and Hindi worth doing.

Finally, the project applies **token economics**. The cost of a support conversation is estimated from input and output token counts and the model's per-token prices (cost = input tokens × input price + output tokens × output price), within the model's context limit. The three approaches change this cost in different ways: RAG adds retrieved passages to every input, a long prompt repeats instructions in every call, and a fine-tuned model can use shorter prompts but has a one-time training cost.

The project will build small prototypes of all three approaches on the same set of sample support queries in English, Telugu and Hindi. They will be compared on answer accuracy, hallucination and failure rate, token usage, estimated cost per query and response behaviour under rate limits. The expected outcome is a clear, evidence-based comparison showing

when each approach, or a combination, is the most suitable for customer support automation.

2. Problem Statement

Companies receive large numbers of customer queries about orders, refunds, billing and product use, and answering them by hand is slow and costly. LLMs can automate much of this work, but a general model does not know a company's policies, can be out of date because of its knowledge cutoff, and may hallucinate answers that mislead customers. Teams must also stay within pricing, rate limits and context limits.

There are three common ways to adapt an LLM: Prompting, RAG and Fine-Tuning. Each differs in accuracy, cost, effort and maintenance, and there is little clear guidance on which one to choose. The problem is made harder in India, where customers write in English, Telugu and Hindi, and the tokenization of these languages changes token counts and cost. This project addresses the question: **which approach gives the most accurate, reliable and affordable customer support automation across these languages?**

3. Objectives

1. To understand the Generative AI landscape, LLMs and next-token prediction, and their capabilities and limits in customer support.

2. To study hallucinations, knowledge cutoffs, pricing, rate limits and failure rates, and how each affects support automation.

3. To implement Prompting, RAG and Fine-Tuning on the same set of customer support queries.

4. To compare tokenization (BPE, WordPiece, SentencePiece) for English, Telugu and Hindi support queries.

5. To estimate cost per query using input and output token counts, context limits and model pricing.

6. To recommend which approach, or combination of approaches, suits different customer support needs.

4. Proposed Methodology

1. Study and Data Collection: Review Module I concepts and prepare a set of sample support queries and a small knowledge base (FAQs, policies, product details) in English, Telugu and Hindi.

2. Tokenization Analysis: Tokenize the same queries with BPE, WordPiece and SentencePiece style tokenizers, and record token counts and IDs for each language.

3. Prompting Approach: Build a baseline using a closed or open-weight LLM with a carefully designed system prompt and a few examples.

4. RAG Approach: Split the knowledge base into chunks, convert them to embeddings, store them in a vector database, and add the retrieved passages to the prompt.

5. Fine-Tuning Approach: Fine-tune a small open-weight model on question-answer pairs from the support data, using an efficient method such as LoRA.

6. Evaluation: Run all three on the same test queries and measure answer accuracy, hallucination rate, failure rate, response time and token usage.

7. Cost Estimation: Calculate cost per query from input and output tokens and per-token prices, including retrieved context in RAG and training cost in fine-tuning.

8. Comparison and Recommendation: Compare the results across languages and approaches, and state which option suits which support scenario.

5. Hardware and Software Required

Hardware

Component	Requirement
Computer	Laptop or desktop with Intel i5 or higher, 8 GB RAM minimum (16 GB recommended)
Storage	At least 20 GB free space for datasets, models and libraries
GPU	Free cloud GPU (Google Colab or Kaggle) for fine-tuning
Network	Stable internet connection for APIs and cloud notebooks

Software

Tool / Platform	Purpose
Python 3	Main programming language
Jupyter Notebook / Google Colab	Writing and running experiments
Hugging Face Transformers and Tokenizers	Open-weight models, tokenizers and fine-tuning
tiktoken, SentencePiece	Token counting for BPE and SentencePiece

LLM API (closed model)	Prompting and RAG with a hosted model
FAISS or ChromaDB	Vector database for RAG retrieval
Sentence embedding model	Converting support documents to embeddings
PEFT / LoRA	Low-cost fine-tuning of an open-weight model
Pandas, Matplotlib	Data handling, cost tables and result charts

6. Expected Outcome

1. A working comparison of Prompting, RAG and Fine-Tuning for a customer support assistant.

2. Clear evidence of how each approach handles hallucinations, outdated knowledge and failures.

3. A token analysis showing how English, Telugu and Hindi differ in token counts and cost.

4. A cost estimate per query and per conversation for each approach.

5. A practical guideline on when to use Prompting, RAG, Fine-Tuning or a combination for customer support automation.

6. A better understanding of Module I topics: LLMs, tokenization, limits of models and token economics.